Python Programming Basics

A beginner-friendly mini guide

A concise, original learning guide with practical examples and a mini project.

Designed for beginners • 3 learning pages + cover

1. Getting Started with Python

Python is a readable, general-purpose programming language used in automation, web development, data analysis, artificial intelligence, and scripting. Its syntax is designed to be approachable for beginners.

Your first program

```
print("Hello, world!")
```

The print() function displays information. Python statements usually do not need semicolons, and indentation is meaningful: it defines blocks of code.

Variables and basic types

```
name = "Asha"       # string\nage = 21            # integer\nprice = 49.50       # float\nis_active =
 True     # boolean
```

A variable is a name that refers to a value. Python determines the type at runtime. Use clear names such as total_price rather than unclear names such as x1.

Input and conversion

```
name = input("Your name: ")\nage = int(input("Your age: "))\nprint(f"Hello {name}, age {age}")
```

input() returns text. Convert it with int() or float() when you need a number. Invalid input can raise an exception, so real applications should validate user input.

2. Decisions, Loops, and Collections

Conditions

```
score = 78\nif score >= 80:\n    print("Great")\nelif score >= 50:\n    print("Passed")\nelse:\n    pri
nt("Try again")
```

Use == to compare values and = to assign them. Conditions can combine with and, or, and not.

Loops

```
for number in range(1, 4):\n    print(number)\n\ncount = 3\nwhile count > 0:\n    print(count)\n    cou
nt -= 1
```

A for loop iterates over items or a range. A while loop repeats while its condition is true. Ensure while loops eventually stop.

Lists and dictionaries

```
languages = ["Python", "JavaScript"]\nlanguages.append("TypeScript")\n\nstudent = {"name": "Asha", "sco
re": 90}\nprint(student["name"])
```

Lists preserve order and can be changed. Dictionaries store key-value pairs and are useful for representing structured records.

Try it

Create a list of five numbers, print each number, and calculate the total with sum(numbers). Then create a dictionary describing a book with title, author, and year.

3. Functions, Files, and Good Practices

Functions

```
def add(a, b):\n    return a + b\n\nresult = add(4, 6)\nprint(result)
```

Functions package reusable logic. Parameters are inputs, and return sends a result back to the caller. Prefer small functions with descriptive names.

Handling errors

```
try:\n    number = int(input("Enter a number: "))\n    print(100 / number)\nexcept ValueError:\n    pri
nt("Please enter a whole number.")\nexcept ZeroDivisionError:\n    print("Number must not be zero.")
```

Catch errors you can handle meaningfully. Avoid hiding every error with a broad except clause, because it makes bugs harder to diagnose.

Reading a text file

```
with open("notes.txt", "r", encoding="utf-8") as file:\n    content = file.read()\nprint(content)
```

The with statement closes the file automatically. Use encoding='utf-8' for predictable text handling.

Next steps

Practice by building a calculator, to-do list, CSV report, or small API client. Learn modules, virtual environments, testing, and package installation as your projects grow.

> Quick reminder: write code, run it, read the error message, and make one small improvement at a time.